A Technical Report on Training RetNet-272M

RetNet-272M Training Technical Report

RetNet Project

2026-10-11

Model	272.30M parameters, 20 layers, 2048-token context
Step time	2176 ms $\rightarrow$ 662 ms ($3.29\times$)
Hardware	RTX 4070 Laptop 8 GB / RTX 3070 8 GB
Data	552.3 M-token packed supervised-fine-tuning corpus

This is the English edition; the Chinese edition is a separate file, RetNet-272M-Chinese-Report.pdf.

Abstract

We present a technical report on the training of **RetNet-272M**, a 272.30M-parameter retentive language model with gated linear attention, trained on a 552.3M-token packed supervised-fine-tuning corpus at a 2048-token context length on a single 8 GB consumer GPU. The report covers:

1. the architecture, including the multi-scale retention, the learnable per-head decay gate, and a function-preserving width-growth layer norm;
2. the data pipeline and its measured composition (7.8 samples per packed row, 2.92% padding, 5.60% of samples carrying chain-of-thought);
3. a set of **engineering optimizations that reduce the per-optimization-step time from 2176 ms to 662 ms (**$3.29\times$**)** under the real `Trainer` path, together with kernel-level attribution showing that the residual step splits 66.4% GEMM / 33.6% elementwise and that the GEMM half already runs at or above its shape-limited ceiling;
4. a closed-form replacement for the chunkwise state scan that cuts it from 1127 µs to 190 µs per layer while remaining within 10^{-6} relative error of the reference recurrence; and
5. three **silent correctness defects** we found and fixed — a `torch.compile` state-dict prefix that made every saved checkpoint unloadable, a resume path that had never worked, and an equivalence test that compared a function against itself.

We also report negative results: fused linear cross-entropy (-13% throughput), a group-norm layout rewrite (-9.2%), `max-autotune` compilation (0%), and physical batch size 2 (memory-infeasible). Finally, one honest negative result: after controlling for the benchmark's budget-truncation confound, the 10,000-step continuation scores **28% lower than its own starting point** on the same 242 never-truncated items ($0.194 \to 0.141$, $\approx 1.6\sigma$), while its stopping behaviour *improved* (truncation $72.5\% \to 65.2\%$, raw think-leak $52.0\% \to 46.8\%$). Our leading — **unverified** — hypothesis is catastrophic forgetting from restarting a full cosine schedule on weights already annealed to `min_lr` (peak LR jumps $3.6\times$): the signature is falling training loss with falling task accuracy.

Keywords: retentive network (RetNet); gated linear attention (GLA); chunkwise parallelism; closed-form state scan; `torch.compile`; 8-bit AdamW; single 8 GB GPU training; silent correctness defects.

Contents

Abstract **2**

1 Introduction **5**

2 Background **5**

3 Architecture **5**
3.1 Configuration . . . 5
3.2 Multi-Scale Retention . . . 6
3.3 The Log-Space Decay Mask: A Real Numerical Trap . . . 6
3.4 Chunked Recurrence and the Closed-Form Scan . . . 7
3.4.1 Reference Chunkwise Form . . . 7
3.4.2 The Key Observation . . . 7
3.5 An Equivalent Form of GroupNorm . . . 8
3.6 Function-Preserving Width Growth (GrowthLayerNorm) . . . 8
3.7 Loss . . . 9

4 Data **9**
4.1 Measured Composition . . . 9
4.2 Conversation Encoding . . . 9
4.3 Side Effects of Packing . . . 10

5 Training Pipeline **10**
5.1 Configuration . . . 10
5.2 Training Legs . . . 10
5.3 A Silent Correctness Bug We Stepped On: HF's Loss Normalisation . . . 12

6 Engineering Optimizations **12**
6.1 Measurement Methodology . . . 12
6.2 Results . . . 13
6.3 Kernel-Level Attribution . . . 13
6.4 Launch-Count Optimization of the Decay Tables . . . 14
6.5 Fixing Two Silent `torch.compile` Failures . . . 14
6.6 Three Dead Config Fields . . . 15
6.7 How Far GEMM Is from the Hardware Ceiling . . . 16
6.8 Negative Results . . . 17

7 Correctness Defects **18**
7.1 The `_orig_mod.` Prefix: No Checkpoint Would Load . . . 18
7.2 The Resume Path Never Worked . . . 19
7.3 An Equivalence Test That Compared a Function With Itself . . . 19

8 Evaluation **20**
8.1 Benchmark Composition 20
8.2 A Metric Defect That Had To Be Written Down 21
8.3 The Completed Comparison 22
8.4 A Causal Hypothesis Awaiting Test: Restarting the LR After Annealing 23

9 Limitations and Future Work **23**

10 Conclusion **23**

A. Appendix A: Reproduction **25**

B. Appendix B: Key Code **25**
B.1 Closed-Form State Scan (`retnet_hf/retention_fast.py`) 25
B.2 Log-Space Decay Mask (`retnet_hf/retention_fast.py`) 26
B.3 In-Place Compilation (`retnet_hf/train.py`) 26
B.4 Export Self-Check (`retnet_hf/train.py`) 26
B.5 Resume Path (`retnet_hf/train.py`) 26

C. Appendix C: External Flag Control (`retnet_hf/control.py`) **27**

D. Appendix D: Defect Summary **27**

1. References **27**

2. Reference Reliability Note **29**

1 Introduction

Our goal is to train a usable Chinese dialogue model end-to-end —pretraining, supervised fine-tuning, evaluation, inference —on a **single 8 GB consumer GPU**. That constraint drives every design decision: parameter count, context length, optimizer, data packing, and which optimizations are worth attempting.

RetNet is attractive precisely here. It replaces the quadratic attention with a linear recurrence while preserving the property of *one model, three equivalent computations*: chunkwise-parallel for training, parallel for long-sequence prefill, and recurrent for token-by-token decoding. The three forms are mathematically identical, so no separate set of inference weights is needed.

This report does not propose a new architecture. It records the **complete engineering facts of one model trained to date**: what the architecture is, what the data looks like, where each step's time actually goes, which optimizations really work, and where things were silently wrong. We consider the last category as important as the first: the three correctness defects reported here share a single property —**none of them raised an error**.

2 Background

RetNet [1] replaces attention with *retention*: a causal, exponentially-decayed linear operator with a parallel form, a recurrent form with an $O(1)$ state (≈ 5.2 MiB total for the whole 272M model), and a chunkwise form used for training. Our implementation adopts the **GLA** [2] parameterisation — a learnable per-head $\log \gamma$ — and the measured gate range is 0.6680–1.0117.

The attraction of retention is threefold. First, the parallel, recurrent and chunkwise forms compute the same function, so one set of weights serves training, long-sequence prefill and token-by-token decoding, and no separate inference model has to be maintained. Second, the recurrent state has a size independent of sequence length: the whole 272M model's layer states together occupy about 5.2 MiB, which is what makes inference on an 8 GB device comfortable. Third, because the decay is a per-head scalar, the gate parameterisation costs almost nothing in parameters while giving each head an independent memory horizon.

The measured gate range matters more than it looks. Heads with $\gamma > 1$ *amplify* history rather than attenuating it, which makes the naive log-space factorisation of the decay mask overflow in fp32, and it spreads the dynamic range of the state across more than a dozen orders of magnitude. Several of the numerical hazards discussed later — the overflowing intra-chunk mask, the choice of a closed-form scan, and the relative rather than absolute criterion in the equivalence test — are direct consequences of that range.

3 Architecture

3.1 Configuration

Table 1. Model configuration

Item	Value
Parameters	**272.30 M** (272,304,448; reconciled digit by digit against `docs/12`)
Layers `num_layers`	20
`d_model`	1024
`num_heads`	16
`key_dim` = `d_model`/`heads`	64
`value_dim`	2048
`value_head_dim` = `value_dim`/`heads`	128
`ffn_dim`	2048
`vocab_size`	20000
`use_swiglu`	false
`use_gla`	**true**
`use_growth_ln`	**true** (`growth_old_dim` = 768)
`tie_weights`	true (`lm_head` and `embedding` are shared)
`embed_scale`	$27.712812921102035 = \sqrt{768}$
`norm_eps`	1e-5
`prompt_loss_weight`	0.1
Context length	2048

Note. `embed_scale` $= \sqrt{768}$ rather than $\sqrt{1024}$: the model grew from width 768 to width 1024 (Section 3.6), and the scaling factor keeps the pre-growth width so that the embedding scale stays consistent with early training.

3.2 Multi-Scale Retention

Each `MultiScaleRetention` layer contains five projections $W_Q, W_K \in \mathbb{R}^{1024\times1024}$, $W_V, W_G \in \mathbb{R}^{1024\times2048}$, $W_O \in \mathbb{R}^{2048\times1024}$, one `GroupNorm`, and a per-head γ. The forward pass is

$$\begin{aligned} q, k, v, g &= W_Q x,\ W_K x,\ W_V x,\ W_G x \\ o &= \text{Retention}(q, k, v; \gamma) \\ o &= \text{GroupNorm}(o) \odot \text{SiLU}(g) \\ y &= W_O\, o \end{aligned}$$

where the g branch acts as a gate, as in GLA [2].

3.3 The Log-Space Decay Mask: A Real Numerical Trap

The intra-chunk mask needs $\gamma^{\,a-b}$ (a, b are intra-chunk indices). The most natural way to write it is the **factorised** form:

$$\gamma^{\,a-b} = \gamma^{\,a} \cdot \gamma^{-b}$$

This form overflows. The smallest gate in this model is $\gamma = 0.668$, and the largest intra-chunk index difference is $C - 1 = 255$ (with $C = 256$):

$$0.668^{-255} \approx 1.0 \times 10^{44} \;>\; 3.4 \times 10^{38} = \text{fp32 maximum}$$

so the result is Inf and the forward output is NaN. `torch.where` **cannot save it** — it only discards the result, it does not prevent the Inf from being computed.

The correct form is **log-space**:

$$\text{mask}[a,b] = \exp\big(\log\gamma \cdot \max(a-b,\,0)\big) \cdot \mathbb{1}[a \geq b]$$

It saturates correctly in both directions: for $\gamma < 1$ the exponent is negative and it underflows to 0 (which is where 0 belongs); for $\gamma > 1$ it gives a finite value. The $\gamma^{\,a-b}$ of the reference implementation happens to be safe (it can only underflow, never overflow), but it cannot be batched into a matrix multiplication.

> **Note. Lesson (now written into the code comments).** An equivalence test must cover the **real gate range**. The first version of the test used only the default initialisation ($\gamma \approx 0.97$–1.0) and could not detect this 10^{44} overflow at all.

3.4 Chunked Recurrence and the Closed-Form Scan

3.4.1 Reference Chunkwise Form

Let the chunk size be C and the number of chunks $n_c = L/C$, with chunk-level decay $d_h = \gamma_h^C$. Write S_i for the state entering chunk i:

$$\begin{aligned}
U_i &= K_i^\top \big(V_i \odot \gamma^{\,C-1-b}\big) && \text{the chunk's contribution to the state}\\
S_{i+1} &= d\,S_i + U_i && \text{state recurrence}\\
\text{inner}_i &= \big(Q_i K_i^\top \odot \text{mask}\big) V_i && \text{intra-chunk}\\
\text{cross}_i &= \big(Q_i \odot \gamma^{\,a+1}\big) S_i && \text{inter-chunk}
\end{aligned}$$

The reference implementation executes the third and fourth lines plus the state recurrence chunk by chunk, emitting about $3n_c \approx 97$ mutually dependent small kernels per layer at $C = 64$.

3.4.2 The Key Observation

d **is constant throughout the loop** (it is the per-head scalar $d_h = \gamma_h^C$, which does not vary with i). The recurrence therefore has a closed form:

$$S_i \;=\; d^{\,i} S_0 \;+\; \sum_{j<i} d^{\,i-1-j} U_j$$

Define the matrix $T \in \mathbb{R}^{H\times(n_c+1)\times n_c}$ and the vector $\tau \in \mathbb{R}^{H\times(n_c+1)}$:

$$T[h,i,j] = \begin{cases} d_h^{\,i-1-j} & j \leq i-1 \\ 0 & j \geq i \end{cases}, \qquad \tau[h,i] = d_h^{\,i}$$

Flattening U into $U_{\text{flat}} \in \mathbb{R}^{B\times H\times n_c\times(d_k d_v)}$, **one batched matrix multiplication** produces every historical state:

$$S_{\text{all}} = T\,U_{\text{flat}} \;+\; \tau \odot S_0 \;\in\; \mathbb{R}^{B\times H\times(n_c+1)\times(d_k d_v)}$$

The first n_c rows are exactly $\{S_0, S_1, \ldots, S_{n_c-1}\}$, and **row** n_c **is exactly the new post-recurrence state** S_{n_c} — no extra recurrence step is needed.

Numerical safety. Every exponent of T, $i-1-j \geq 0$, is **non-negative**, so the γ^{-j} overflow of Section 3.3 cannot occur here. However small d is, it can only underflow to 0, and that correctly means "the history has been forgotten".

Cost analysis. The FLOPs rise from $O(n_c H d_k d_v)$ to $O(n_c^2 H d_k d_v)$ — at $C = 128$ ($n_c = 16$) that is $17 \times 16 \times 8192 \times 16 \approx 3.6 \times 10^7$ MAC, i.e. 71 MFLOP, entirely negligible against the 3.46 TFLOP of GEMM per micro-step. What is bought is the **kernel launch count**: 97 $\rightarrow$ about 5.

Measured (eager, one layer, $C = 64$, $n_c = 32$):

Table 2. Per-layer cost of the closed-form scan

Implementation	Per-layer time	$\times$ 20 layers
Chunk-by-chunk loop + `torch.stack`	**1127.5** μs	22.6 ms/micro-step
Closed-form batched matmul	**190.5** μs	3.8 ms/micro-step

Equivalence. Against the reference recurrence it is **bit-identical** for $\gamma \leq 0.85$; for $\gamma \geq 1$ there is a 1–3% relative difference whose source is a different rounding path — the old form multiplies the bf16 d sixteen times in a row (accumulated error $\approx 15 \times 2^{-8} \approx 6\%$), while the new form computes d^k in fp32 and converts once ($\approx 0.4\%$). **Both are at the level of fp32 accumulation-order noise, and the new form is the more accurate of the two.** With the criterion fixed, all 15 combinations in `tests/test_fast_chunkwise.py` pass (relative error $\sim 10^{-6}$).

3.5 An Equivalent Form of GroupNorm

`MultiScaleRetention` uses `nn.GroupNorm(num_groups=H, num_channels=value_dim)` applied to $[B{\cdot}L,\ H{\cdot}v_{hd},\ 1]$. Because **the number of groups equals the number of heads** and the spatial dimension is 1, each group is exactly the v_{hd} channels of one head, so the operator is **equivalent** to a LayerNorm along the last dimension for each (b, h, l) (biased variance, the same ε), with the affine coefficients grouped per head:

$$y[b,h,l,j] = \frac{x[b,h,l,j] - \mu_{b,h,l}}{\sqrt{\sigma^2_{b,h,l} + \varepsilon}} \ \cdot\ w[h{\cdot}v_{hd} + j] \ +\ b[h{\cdot}v_{hd} + j]$$

We tried rewriting the operator on that basis as "normalise first, then permute once at the end", in order to remove the reference implementation's intermediate copy. **It measured 9.2% *slower*** (190 vs 174 ms per micro-step): the reference implementation puts the permute *before* the normalisation, so that copy is immediately absorbed by the elementwise normalisation kernel; moving it *after* makes it the input of `out_proj` (an external cuBLAS call), where it can no longer be fused into anything. **Reverted**; the test stays in `tests/test_head_group_norm.py`.

3.6 Function-Preserving Width Growth (GrowthLayerNorm)

The 768 $\rightarrow$ 1024 width growth of this model relies on a two-segment LayerNorm. Let the old dimension be d_o, the new dimension d_n, and the total $d = d_o + d_n$:

$$\mathrm{LN}_{\text{grow}}(x) = \left[\, \mathrm{LN}_{\text{old}}(x_{1:d_o}) \,\middle\|\, \gamma_{\text{new}} \odot x_{d_o+1:d} + \beta_{\text{new}} \,\right]$$

With the initialisation $\gamma_{\text{new}} = 0,\ \beta_{\text{new}} = 0$, the output of the new channels is **identically 0**, so the residual branch of that layer is **bit-identical at the instant of growth** — this is strict function-preserving initialisation (FPI) in its LayerNorm form.

The key point is that **the statistics of the old channels are not affected by the new ones**: if a standard LayerNorm is applied directly to the concatenated d-dimensional vector, both μ and σ are polluted by the new channels. Writing it in two segments decouples the two.

3.7 Loss

Weighted by the weight code $w \in \{0, 0.1, 1\}$ and self-normalising:

$$\mathcal{L} = \frac{\sum_i w_i \cdot \mathrm{CE}\big(\mathrm{logits}_i,\ y_i\big)}{\sum_i w_i}, \qquad w_i = \begin{cases} 0 & \text{padding} \\ 1 & \text{assistant reply / plain text} \\ 0.1 & \text{prompt} \end{cases}$$

The prompt weight is 0.1 so that the model conditions on the user input without spending generation capacity on repeating the question.

4 Data

4.1 Measured Composition

A full read-only scan of `data_cache/chat_pack_v13/train.ids.npy` (269,561 $\times$ 2049) together with `train.wcode.npy` gives:

Table 3. Measured composition of the packed corpus

Quantity	Measured
Rows $\times$ width	269,561 $\times$ 2049
Total token slots	**552,330,489**
Padding (wcode $= 0$)	16,126,035 (**2.92%**)
Weight-code distribution	reply 358,896,326 / prompt 144,461,792 / plain 32,846,336
Samples (`<bos>` count)	**2,107,335** (7.8 per row on average)
Rows with `<think>` / rows with `</think>`	80,504 / 69,199 (row-level closure **86.0%**)
`<think>` occurrences / `</think>` occurrences	123,604 / 100,040 (occurrence-level closure **80.9%**)
Share of samples carrying `<think>`	**117,973 / 2,107,335 = 5.60%**

4.2 Conversation Encoding

`retnet/chat.py` is the single implementation **shared** by data construction and inference, which guarantees that the boundaries agree:

```
<bos> [设定：<persona>\n] 用户：U1\n助手：<think>R1</think>A1 [\n用户：U2\n助手：⋯] <eos>
```

`<think>` = `[5690, 533, 2605, 33]` and `</think>` = `[5690, 18, 533, 2605, 33]` — **differing by exactly one token** (id 18 = `/`). Decoded id by id: `5690='<'`, `533='th'`, `2605='ink'`, `33='>'`, `18='/'`.

Three corollaries:

1. **Closure is a binary decision to insert one token at one position**, and is therefore intrinsically capacity-independent.
2. They are **ordinary BPE tokens, not registered special tokens**, so `skip_special_tokens=True` does not strip them and a leak is visible text in the decoded output.

3. `<` = 5690 (no leading space) and 19965 (`>` with a leading space) are **different tokens**; the same holds for `/` = 18 and 1788. A character-level criterion must therefore be written on ids, not on strings.

4.3 Side Effects of Packing

`build_chat_mix_v13.py` is `concat([v9 re-cut rows, newly encoded rows])`, with the boundary at $269{,}192{,}388//2049 = 131{,}377$ rows:

- base region: 131,377 rows, **padding = 0**;
- newly encoded region: 138,184 rows, **padding = 16,126,035 (5.7%)** — all of the padding lives here.

The mechanism: a truncated sample is exactly 2048 tokens long, so it can only occupy a row of its own; best-fit packing has nothing that fits beside it. The cost is 16.1M slots (2.9% of the whole pack) thrown away — they carry $w = 0$, so they do not enter the loss, but they do occupy context window.

5 Training Pipeline

5.1 Configuration

Table 4. Training configuration

Item	Value	Note
`batch_size`	1	physical batch, limited by the 8 GB card
`grad_accum`	4	effective batch 4 = 8192 tokens/optimization step
`chunk_size`	64 (first leg) / **128** (second leg)	see Section 6.6
`max_steps`	50,000	optimization steps
`lr` → `min_lr`	4e-5 → 4e-6	`cosine_with_min_lr`
`warmup`	500	
`optim`	`adamw_bnb_8bit`	8-bit AdamW [11]
`precision`	bf16	
`grad_checkpoint`	**false**	turning it off saves 1/3 of the step time, at the cost of resident activations
`compile`	**true**	`torch.compile(inductor)`, see Section 6.5
`save_steps` / `eval_steps`	2500 / 2500	
`eval_batch_mult`	4 (first leg) → **1** (second leg)	see Section 6.6
`auto_restart_max`	3	see Section 7.2

5.2 Training Legs

Table 5. Training legs

Leg	Step range	Status	Note
First leg	0 → 10,101	completed	continuation exported from v13; `chunk_size` was in fact 64
Second leg	10,000 → 50,000	**in progress**	continuation from `checkpoint-10000`, `chunk_size=128`

The first leg's measured loss (recorded every 20 steps) falls from **2.4861** to **2.3707**; the last `learning_rate = 3.6827e-05` agrees exactly with the configured cosine schedule's **theoretical value 3.68e-05 at step 10100**,

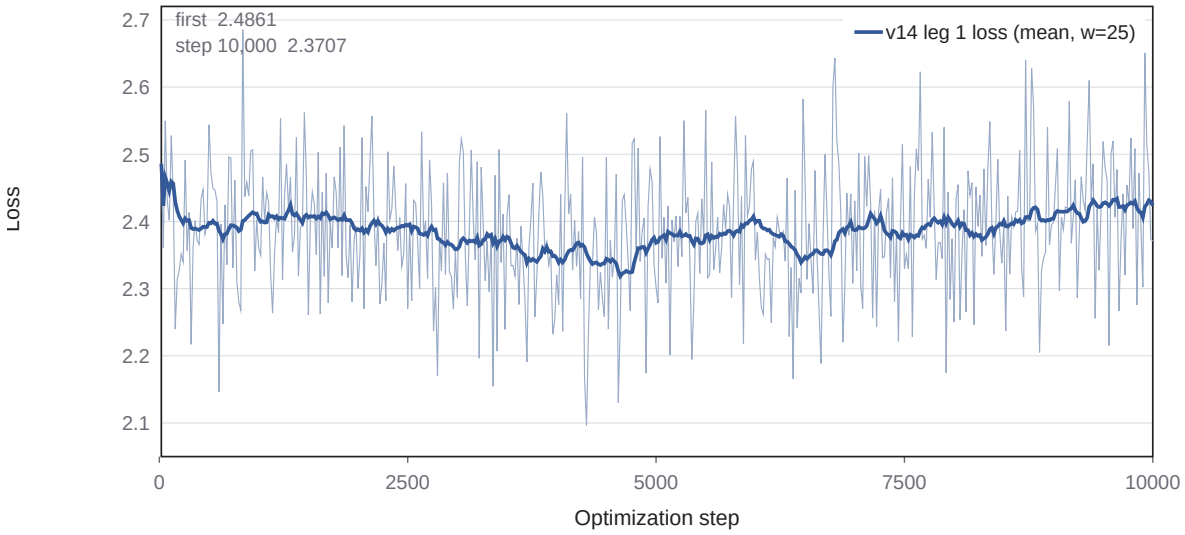


Figure 1. Training loss of v14 leg 1 (light: per-step, dark: 25-step rolling mean). Measured $2.4861 \rightarrow 2.3707$.

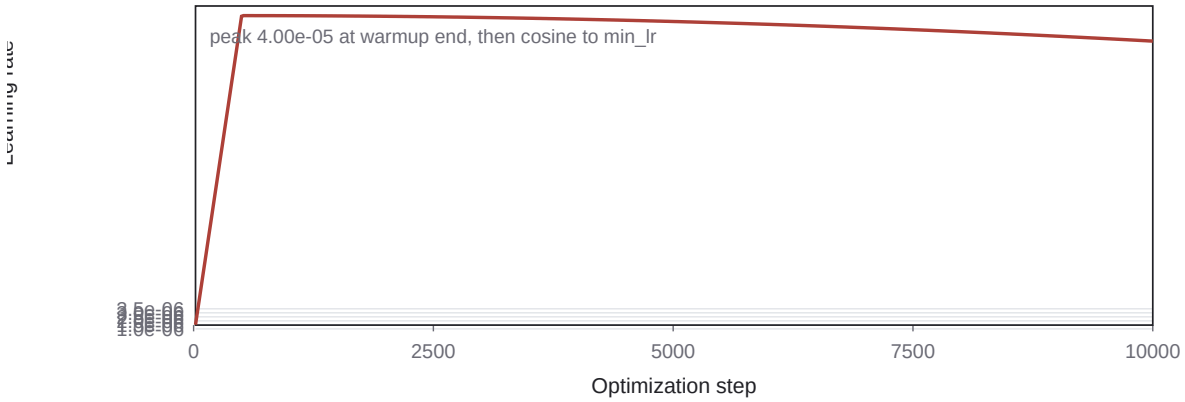


Figure 2. Learning-rate schedule over the same interval: warmup to a peak of 1.52×10^{-6}, then cosine decay.

which shows that both the scheduler and the progress accounting are correct.

The second leg (the continuation, `chunk_size=128`) reproduces the same behaviour over the same interval:

```
首条  {'loss': 2.4092, 'grad_norm': 2.739, 'learning_rate': 3.681386e-05, 'epoch': 0.15}
末条  {'loss': 2.3686, 'grad_norm': 3.412, 'learning_rate': 3.574538e-05, 'epoch': 0.17}
```

The LR falls monotonically from 3.6814e-05 to 3.5745e-05, **exactly the trajectory cosine should follow**; the loss fluctuates between 2.23 and 2.68, the same range as the first leg over the same interval. This simultaneously verifies that **the optimizer state, the LR-scheduler state and the global step count are all restored correctly by HF**.

> **Note.** There is one unexplained observation: the first leg's loss near step 10020 is about 2.47, while the second leg at the same position is 2.4092 — a gap of about 0.06, far larger than the measured chunk-64/128 single-batch difference of 2.4e-7 reported in Section 6.6. The second leg was rolled back to `checkpoint-10000` and rerun (the 101 steps 10001–10101 of the first leg were discarded), so the two should have seen the same data window. **We did not get to the bottom of this**; it is listed as an open item in Section 9.

5.3 A Silent Correctness Bug We Stepped On: HF's Loss Normalisation

The trailing `**_kwargs` in the signature of `RetNetForCausalLM.forward` makes HF's heuristic infer `accepts_loss_kwargs = True`, and `Trainer.training_step` therefore **skips**

```
loss = loss / self.current_gradient_accumulation_steps
```

while this model's loss is already self-normalised by $\sum w$ (it does not use `num_items_in_batch`). The consequence is that **the gradients are inflated by a factor of `accum` and the effective learning rate rises by the same factor** — silently, and only when `accum > 1`. The measured symptom was the loss going from 3.19 to **12.75** $\approx$ **3.19** $\times$ **4**. The fix is to declare it explicitly on the model class:

```
accepts_loss_kwargs = False     # note: the attribute name has no model_ prefix
```

After the fix the loss immediately returns to 3.1851 $\rightarrow$ 3.0148, consistent with 3.20 $\rightarrow$ 3.17 at a real batch of 4.

> **Note.** Every previous task in this project used `accum = 1`, so this mine was never stepped on. **The moment gradient accumulation is enabled it has to be fixed first.**

6 Engineering Optimizations

6.1 Measurement Methodology

All step times are measured with `scripts/bench_trainer.py`, which **replicates** the entire path of `run_task.py` (same model construction, same `TrainingArguments`, same dataset object, same collator), discards the first 25 steps (including inductor's first compilation), and takes **four times the median micro-step** as the optimization-step time.

Three disciplines, each of which was bought with a mistake:

1. **Wall clock rules everything.** The profiler's **operator attribution misleads**: it credited `NativeDropoutBackward0` with 25%, while actually turning dropout off saved only 3.9%.
2. **The power source must always be stated.** The same configuration is about 40% slower on battery; an early "5.9% rotary-cache gain" was a phantom measured that way. Every number in this report was measured at `BatteryStatus=2` (AC).

3. **Never tune the algorithm without watching memory.** Disabling gradient checkpointing while memory has not been freed causes thrashing and presents as "the algorithm got slower" — the same configuration was measured at 1437 ms and then at 183 ms purely as a result of memory.

6.2 Results

Table 6. Optimization step time and throughput by configuration

Configuration	Step time	Throughput	vs A
A reference: chunk loop, chunk 64, batch 4, gradient checkpointing, no compile	2176 ms	3764 tok/s	1.00×
B batched chunkwise + `torch.compile`, batch 4, gradient checkpointing	903 ms	8814 tok/s	**2.41**×
C = B + batch1×accum4 + no gradient checkpointing + 8-bit AdamW	713 ms	11496 tok/s	**3.05**×
C′ = C + closed-form scan + decay-table reuse (this report)	**662 ms**	**12373 tok/s**	**3.29**×

C′ is **1.36**× relative to B and 1.08× relative to C. Three repeat measurements: 657 / 662 / 663 ms, jitter $< 1\%$.

The mechanism of each step:

- **A → B (2.41×)**: the four small matmuls per chunk ($C = 64$ means 128 per layer per step, 9241 `aten::matmul` model-wide) become three cross-chunk batched matmuls plus one small state scan, dropping the number of `matmul` calls to 441.
- **B → C (1.26×)**: gradient checkpointing adds a whole extra forward pass — forward once + recompute once + backward once = 3 passes instead of 2 — so **a full 1/3 of the step time is recomputation**. Removing it requires freeing memory first: 8-bit AdamW compresses the optimizer state from 2.18 GiB to 0.55 GiB, and `batch 1 × accum 4` cuts the activations to a quarter.
- **C → C′ (1.08×)**: see Section 3.4.

6.3 Kernel-Level Attribution

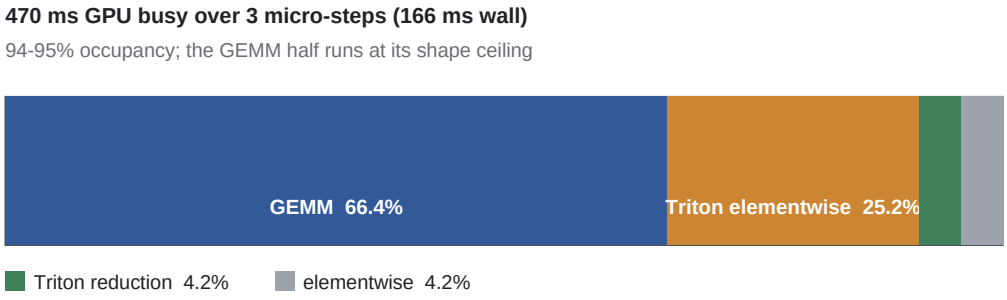


Figure 3. GPU busy time aggregated by CUDA kernel class over three micro-steps at C′.

After exporting a chrome trace, `scripts/profile_kernels.py` aggregates **by CUDA kernel** rather than by operator. The sample below covers 3 micro-steps under C′:

Table 7. Kernel-level busy time at C′ (three micro-steps)

Category	Busy time	Share	Kernels	Mean
GEMM	312 ms	**66.4%**	2169	143.8 μs
Triton elementwise	118 ms	25.2%	3462	34.1 μs
Triton reduction	20 ms	4.2%	555	35.6 μs
`elementwise`	20 ms	4.2%	762	25.7 μs

Total busy time 470 ms / 3 micro-steps = 157 ms against 166 ms of wall clock = 94–95% occupancy.

> **Note. An old conclusion that has been overturned.** The project previously used "GPU busy time / wall clock" to declare configuration C to be **20.8% idle**, and concluded from that that "removing kernels is useless". That figure was measured **on battery**. On AC the true occupancy is **94%** — the premise is false and the conclusion "removing kernels is useless" collapses with it. In fact both optimizations in this report (Section 3.4 and Section 6.4) are kernel removal, and together they recover 8%.

6.4 Launch-Count Optimization of the Decay Tables

`_decay_tables` builds the intra-chunk mask and three 1-D decay tables for every layer. It handles only 260,000 elements, yet it costs **410 μs** — almost all of it **launch latency**: about 22 mutually dependent small kernels at roughly 18 μs per launch on Windows.

The goal is therefore not "compute less" but "launch less":

1. cache the index tensors (`arange`/`sub`/`ge`/`clamp`), saving 4 launches;
2. reuse one γ^j table ($[H, C]$) for all three 1-D tables: cross $= \gamma^j \cdot \gamma$, value $=$ flip(γ^j), chunk $= \gamma^j[-1] \cdot \gamma$, compressing 3 `exp` calls into 1.

Effect: the 469 μs reduction kernel that used to hold a Top-18 slot (`triton_red_fused__to_copy_add_exp_log_mul_select_sum_view_zeros`) **disappears from the list**, and Triton-reduction busy time drops **47 $\rightarrow$ 20 ms**.

6.5 Fixing Two Silent `torch.compile` Failures

Before 2026-10-10, `torch.compile` was **completely ineffective** in this project: measured 0.0%. There were two independent reasons:

1. **No Triton.** The laptop's conda environment was missing `triton-windows`. Inductor compiles **lazily**, so `TritonMissing` is raised only at the first forward pass — long after `train.py`'s `try` block has been left — and it therefore **neither crashes nor speeds anything up: it silently does nothing**.
2. **The compiled model was never called.** `Trainer.__init__` stores the **original** model in `self.model_wrapped`, while `train()` takes `self._wrap_model(self.model_wrapped)`. Replacing only `trainer.model` means the compiled model is never invoked: the log says `torch.compile applied`, the dynamo counters are empty, and the speed does not change at all (it was in fact 1.8$\times$ *slower* because of `OptimizedModule`'s attribute forwarding).

After the fix the dynamo counters show `{'frames': {'total': 1, 'ok': 1}, 'unique_graphs': 1}` — **one whole graph captured**.

> **Note.** But it was exactly this fix — replacing both attributes at once — that introduced the far more serious prefix defect of Section 7.1. The final solution is given there: **replace `forward` in place** instead of wrapping the module.

6.6 Three Dead Config Fields

`config.chunk_size` was always a dead field. In the signature of `RetNetForCausalLM.forward`, `chunk_size=64` is a **function default**, the HF Trainer **never** passes that keyword when it calls the model, and `forward` **never reads** `self.config.chunk_size` (it only `getattr(self.config,"mode")`). Measured:

```
config.chunk_size = 64 / 128 / 256  ->  loss all 2.69759822   (bit-identical = config ignored)
explicit chunk_size = 128           ->  loss 2.69759798       (the parameter itself is live)
```

So the project always ran at chunk 64 while the task JSON, the logs and the docs all said 128. The `--sweep chunk_size=32,64,128,256` of `bench_step.py` swept a variable that did nothing, and the sentence in `docs/15` §6 — "chunk 512/1024 is actually slower, the optimum is 128" — **does not hold**.

The **first valid chunk sweep**, after the fix:

Table 8. Chunk-size sweep

chunk	Step time	reserved
64	663 ms	6.73 GiB
128	**662 ms**	**6.61 GiB**
256	685 ms	6.90 GiB

64 and 128 tie; 256 is 3.5% slower — chunk size is a pipeline-correctness issue, not a performance lever.

`eval_batch_mult` was dead too: the configuration template of `run_task.py` contains it, but it was never passed to the training script, and `train.py` hardcoded `*4`. Measured: with 1×4 the first evaluation costs an extra **128 s** (shape recompilation for the evaluation) and peak reserved reaches **7.65 / 8.00 GiB**, leaving only 0.35 GiB of headroom. With 1×1 the evaluation cost drops from **6.6% to 2.9%** of the round and reserved returns to 6.73 GiB.

`save_total_limit` was dead as well — and this one silently deleted data. `tasks/v14_272m_2k_cont.json` and `tasks/v14_272m_2k_resume.json` both say `save_total_limit: 4`, `run_task.py`'s template has the key but **never builds it into the command line**, and `train.py` hardcodes `save_total_limit=3`. Reading the persisted `checkpoint-12500/training_args.bin` gives the value that actually took effect: **3**. The 4 in four configuration files never once applied.

The consequence is that `checkpoint-10000` was deleted. The chain has three links, each independently verifiable:

1. HF's `_rotate_checkpoints(use_mtime=True, output_dir=run_dir)` (`transformers/trainer.py`, v4.57.6) sorts by **directory mtime, not by step number**, and `shutil.rmtree`s the oldest beyond the limit.
2. When the author repaired the checkpoints at 20:14 with `strip_compile_prefix.py --in-place`, `Get-ChildItem` enumerated in ascending name order, so the rewrite order was `10000` → `5000` → `7500`; **`checkpoint-10000` therefore became the oldest of the three by mtime** (on-disk timestamps: `checkpoint-5000` 20:14:08, `checkpoint-7500` 20:14:09, `10000` earlier).
3. After `checkpoint-12500` was written at 20:59 there were four directories > 3, and HF deleted the first in sort order — exactly that one.

The deletion is completely silent: the default `transformers` log level is `WARNING` (measured `transformers.utils.logging.get_verbosity() == 30`) while the deletion line is `logger.info("Deleting older checkpoint [...] due to args.save_total_limit")`, which is below the threshold, so not one character of it appears in `logs/v14_272m_resume.log`. This also explains why `retnet_272m_v13` and `retnet_272m_v14` each keep only **3** checkpoints.

The three dead fields share one shape: present in the config JSON → not passed by `run_task.py` → default in `train.py`. There is exactly one criterion, and it is cheap: **read the persisted `training_args.bin`**,

never the config JSON. These are the values read that way:

```
save_total_limit = 3            # the config JSON says 4
logging_steps    = 20           # hence one log_history entry every 20 steps
save_steps       = 2500
eval_batch_mult  = <absent>     # not in TrainingArguments; used to be a dead field
```

`logging_steps=20` also retires an observation that had been in doubt: the `log_history` in `checkpoint-10000`'s `trainer_state.json` has length 504, which once looked anomalous; but $10000/20 + 4 = 504$ (four evaluations, one every 2500 steps) and `checkpoint-12500` is $12500/20 + 5 = 630$, digit-for-digit equal to the observed 630 — **it had been right all along**.

Now wired. `run_task.py` passes `save_total_limit` / `save_steps` / `eval_steps` / `weight_decay` / `max_grad_norm` on the command line, and `train.py` declares matching arguments (with defaults equal to the old hardcoded values, so behaviour changes only when the config really differs). The verification is that `run_task.py --config ... --dry` prints `--save_total_limit 4` in the command line, plus an AST check that every `args.*` in `train.py` has a corresponding `add_argument`.

One sibling is not yet wired: `--eval_batches 40` is read only by `train.py:429`'s `evaluate()` (the non-HF path), while the HF `Trainer` path evaluates the entire validation set — consistent with an `eval_runtime` of 95.5 s. This item is **static review only, not measured**.

6.7 How Far GEMM Is from the Hardware Ceiling

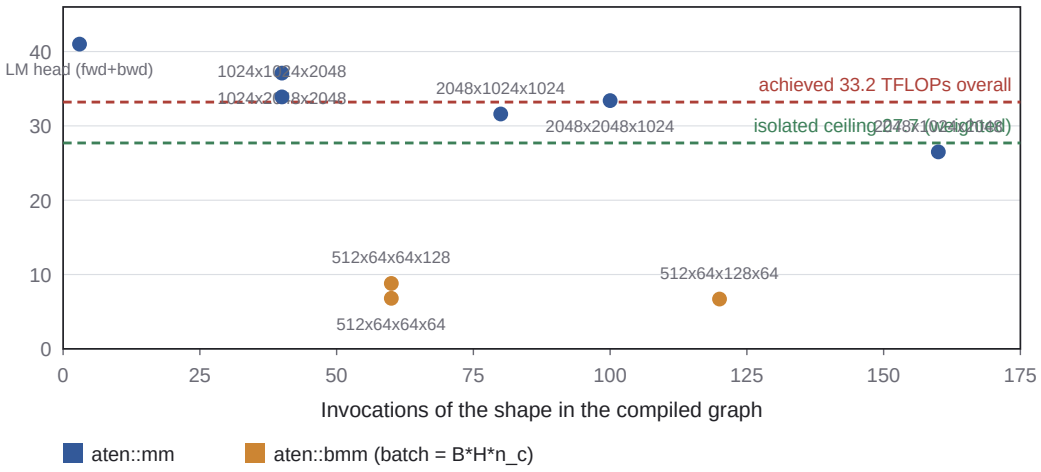


Figure 4. Matmul shapes issued by the compiled graph, with their in-place measured peaks and two reference lines.

First, correct a wrong number that has been cited repeatedly. The project's docs claim "11–21 TFLOPs for batch-1 shapes", but the measurement is **26–42** (table below). Using the batch-4 large-grid 42 TFLOPs as a batch-1 ceiling manufactures a phantom "40% headroom".

`scripts/gemm_ceiling.py` computes an exact floor from two measured inputs: the shape list comes directly from the compiled graph's dynamo counter **`aten_mm_info`** (the `aten::mm` / `aten::bmm` calls that graph actually issued; the counts are self-consistent with the LM head's three passes — forward and backward — at 1 each), and each shape's peak is measured in place **without synchronising between calls** (otherwise the

0.1 ms chunkwise batched matmuls are drowned by sync overhead, under-reporting by 3–10×).

Table 9. Matmul shapes issued by the compiled graph

Kind	Shape	Count	TFLOPs	Subtotal ms
mm	2048×1024×2048	160	26.5	51.9
mm	2048×2048×1024	100	33.4	25.7
mm	2048×1024×1024	80	31.6	10.9
mm	1024×2048×2048	40	37.1	9.3
mm	1024×1024×2048	40	33.9	5.1
mm	LM head, three passes	1 each	40–42	6.1
bmm	512×64×128×64	120	**6.7**	9.7
bmm	512×64×64×64	60	**6.8**	2.4
bmm	512×64×64×128	60	**8.8**	3.7

(The bmm batch is $B \cdot H \cdot n_c$; the matrices are only 64×64 or 64×128, so these reach only 6.7–8.8 TFLOPs.)

Conclusion:

Table 10. Measured GEMM busy time against the shape-limited floor

	ms/micro-step
Floor from the measured per-shape peaks	**124.6**
Profiler-measured GEMM busy time	**104.0**
Ratio	**0.83**×

The measurement is below the isolated floor. Put differently: it achieves **33.2 TFLOPs** (3.457 TFLOP ÷ 104 ms), while the weighted shape peak is **27.7 TFLOPs** (3.457 ÷ 124.6 ms) — **already above the ceiling for these shapes**. Both explanations (isolated measurement is conservative / the profiler buckets some matmuls under Triton) point the same way: **there is no GEMM headroom left.**

The remaining 33.6% non-GEMM is about 3500 Triton elementwise/reduction kernels at an average of 34 μs and roughly 350 GB/s — **running at DRAM bandwidth** (measured local copy bandwidth 359–396 GB/s). That part has an arithmetic intensity of only 0.5–2 FLOP/byte, is **batch-invariant**, and can only improve by fusion — which inductor has already exhausted.

6.8 Negative Results

Table 11. Negative results of the attempted optimizations

Attempt	Result	Reason
Fused linear + cross-entropy (Liger-style [12])	**13% slower** (batch 1)	Numerically equivalent (bit-identical loss, gradient relative difference $\leq$ 2.3e-4) and saves 452.7 MiB at batch 2, but the chunked loop requires `torch._dynamo.disable` (otherwise the graph breaks and hangs), which loses compile fusion
`_head_group_norm` layout rewrite	**9.2% slower**	see Section 3.5
`torch.compile(mode="max-autotune-no-cudagraphs")`	**0%** (662 vs 662)	GEMM goes to external cuBLAS; autotune only affects Triton matmul templates
CUDA Graphs (`mode="reduce-overhead"`)	unusable	hard conflict with gradient accumulation (the same graph runs 4 times with separate backwards)
Log-depth (Hillis–Steele) scan	**2.8$\times$ slower**	each round `torch.cat` materialises the whole $[B, H, n_c, d_k, d_v]$
Physical batch 2	memory-infeasible	needs 9.81 GiB reserved against an 8 GB card
Layered gradient checkpointing	loses to C′	$K = 10/20 \rightarrow 816$ ms
`dataloader_num_workers` 0/2/4	**0%**	the data pipeline is not the bottleneck
Removing `logits.float()` from the loss	**0%**	loss is bit-identical; the profiler's 3.9% was an attribution artefact
dropout 0.1 $\rightarrow$ 0.0	only 3.9% saved	and it changes the regularisation strength, so it is not worth it

7 Correctness Defects

This section records three **silent** defects. What they have in common is: **no error is raised, nothing crashes, and the logs look normal.**

7.1 The `_orig_mod.` Prefix: No Checkpoint Would Load

`torch.compile(model)` returns an `OptimizedModule`, which places the original model under an `_orig_mod` submodule, so **every key** of `state_dict()` gains an `_orig_mod.` prefix.

In order to actually enable compilation (Section 6.5), `retnet_hf/train.py` assigned the compiled module to `trainer.model` and `trainer.model_wrapped` at the same time. HF's `save_model` takes `self.model.state_dict()` — **so every checkpoint written out carried the prefix.**

And `RetNetForCausalLM.from_pretrained` loads them **without error**: it discards `_orig_mod.*` as "unexpected keys" and **randomly initialises** `blocks.*` as "missing keys". The result:

```
v13 final / rp / v10      365 keys, 0 prefixed    forward loss = 2.5767   logits finite
v14 checkpoint-10000      365 keys, 365 prefixed  forward loss = nan      all logits nan
```

On the evaluation side the symptom is `torch.multinomial` raising `CUDA error: device-side assert triggered` — **the root cause is simply not reachable from the symptom**.

Regression localisation: v13's configuration had `compile=False`, v14 has `compile=True`. The causal chain closes completely.

The fix (three layers):

1. **At the source**: stop wrapping the module and instead **replace `forward` in place** — `trainer.model.forward = torch.compile(trainer.model.forward)`. Compilation still takes effect

(measured 1.27 it/s, where eager drops to 0.45), while `state_dict` stays clean, so **save, export and resume all become correct at once**.

2. **Rescue**: `scripts/strip_compile_prefix.py`, which strips the prefix in place and keeps a `.orig` backup.
3. **Gate**: `verify_export()` checks the key names after every `save_model` and raises a loud alarm; `scripts/check_checkpoints.py` (binary, $O(1)$) and `scripts/verify_checkpoint_loads.py` (which really runs one forward pass) are available as a pre-load health check.

All 17 historical checkpoints have been repaired and re-verified clean, and 6 bad backups have been deleted.

7.2 The Resume Path Never Worked

`train.py` used to pass `resume_from_checkpoint=args.output_dir`. But HF's `Trainer._load_from_checkpoint` looks for weight files **directly** in the directory it is given (`WEIGHTS_NAME` / `SAFE_WEIGHTS_NAME` / …), while `checkpoint-N` is a **subdirectory** of it, so it necessarily raises:

```
ValueError: Can't find a valid checkpoint at checkpoints/retnet_272m_v14
```

This path had never actually been walked (the v14 log does not contain a single `[restart]` line), so `auto_restart_max: 3` was an **empty promise**: the moment a real crash happened, `run_task.py` would restart, immediately raise the same error again, and exit after three rounds. The scene is:

```
[exit] rc=1
[restart] attempt 1/3 wait 3min
```

The fix: resolve the real checkpoint directory with `transformers.trainer_utils.get_last_checkpoint(output_dir)`, pass that, and print which one was chosen.

> **Note.** This defect has a more dangerous adjacent form: once the checkpoints are repaired clean, resume fails **the other way** — HF does the `load_state_dict` with the compiled module as the target, so all the clean keys are discarded as "unexpected". Without the fix of Section 7.1, training would run **with the checkpoint's optimizer state on top of the initial weights from `model_dir`**, while the log prints only two `missing/unexpected keys` warnings.

7.3 An Equivalence Test That Compared a Function With Itself

`tests/test_fast_chunkwise.py` is designed as: reference implementation vs batched implementation. But `modeling_retnet._chunkwise_retention` **forwards directly to `chunkwise_batched`** when the environment variable `RETNET_FAST_CHUNKWISE=1` — and the training environment always had that variable set.

So the "reference implementation" was the function under test, and the test always reported "bit-identical". The table in `docs/15` §2.3 claiming that "all combinations have bit-identical state" **was never actually verified**.

The fix: add a `reference_path` context manager that forces the sub-module switch, and change the criterion from an **absolute** threshold to a **relative** quantity (heads with $\gamma > 1$ amplify the state to 10^{13}, which makes an absolute threshold of 10^{-4} meaningless).

The real conclusion after the fix (all 15 combinations pass):

Table 12. Equivalence-test results

gamma	out rel_mean	out rel_max	state rel_max
Default init 0.9688–1.0000	∼1.4e-6	2.5e-7 – 6.8e-7	2.4e-7 – 7.0e-7
Real gate 0.6680–1.0117	∼8.1e-7	9.0e-7 – 2.3e-6	5.1e-7 – 2.1e-6
Extreme 0.30–1.02	∼6.2e-7	3.1e-7 – 3.6e-6	9.3e-7 – 2.7e-6

That is: the fast chunkwise path is not bit-equivalent but a relative-error 10^{-6}-level approximation (the same order of magnitude before and after the change; the closed-form scan did not make it worse).

8 Evaluation

8.1 Benchmark Composition

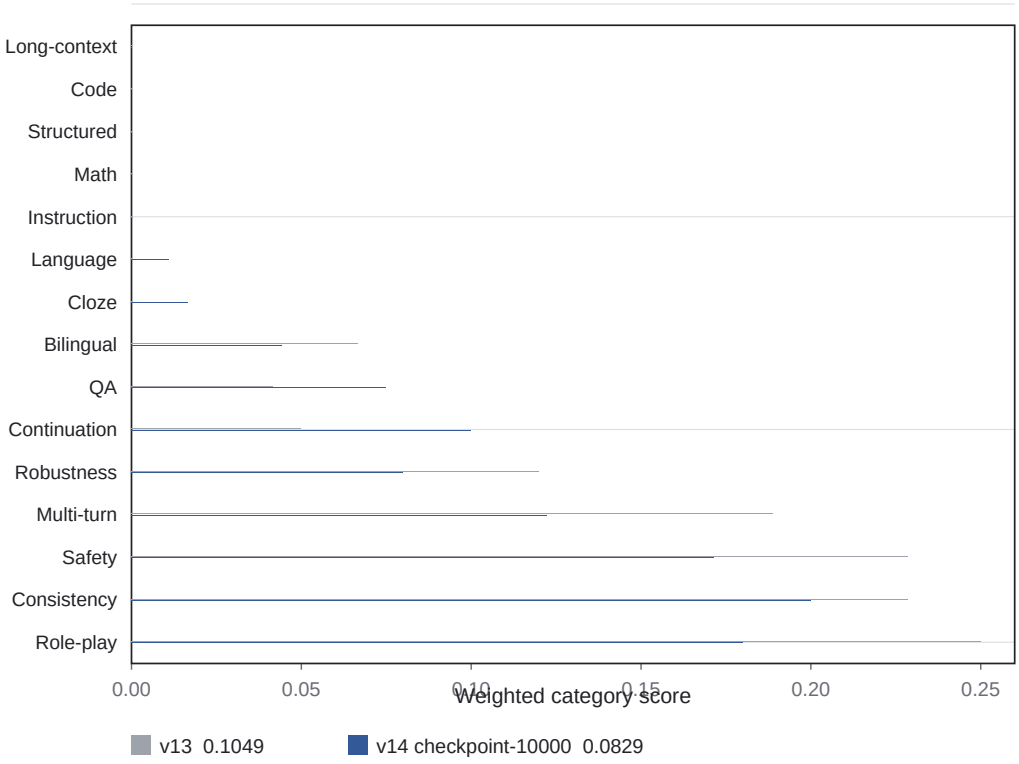


Figure 5. Per-category weighted score for v13 and v14 on the same item bank and seed.

`bench/data/bench_v3.jsonl`: **1,020 items / 15 categories / 136 families**, frozen with a sha256. Decoding parameters $T = 0.7$, top_k 40, top_p 0.9, repetition_penalty 1.1, no_repeat_ngram 4, single seed 42.

Table 13. Benchmark composition by category

Category	Items	Category	Items
Cloze	120	Instruction following	80
QA	120	Consistency	70
Roleplay	100	Structured output	60
Multi-turn	90	Continuation	60
Language	90	Robustness	50
—	—	Long context	50
—	—	Bilingual	45
—	—	Safety	35
—	—	Code	25
—	—	Math	25

8.2 A Metric Defect That Had To Be Written Down

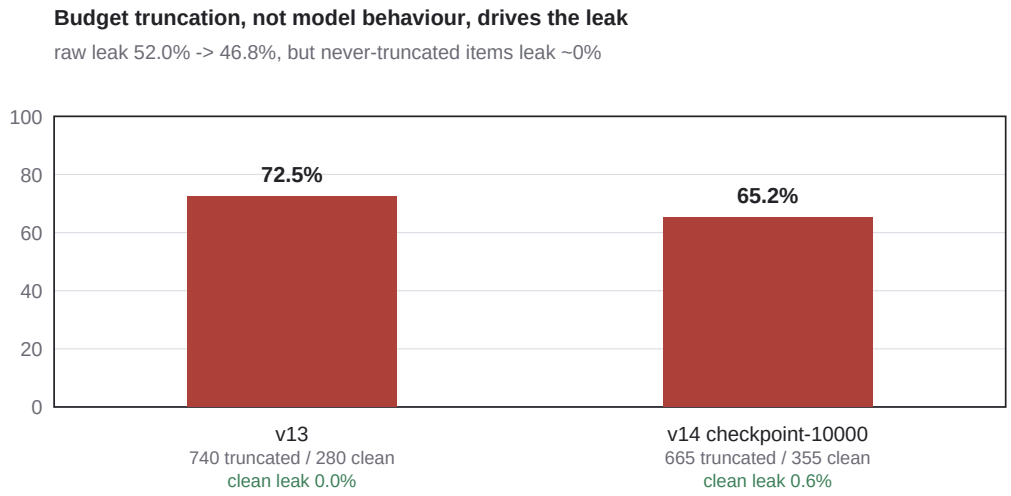


Figure 6. Truncation rate after splitting by budget truncation; never-truncated items leak at essentially zero.

Every item carries its own `max_new`, measured to lie entirely in 16–160 (median 70), whereas the median CoT length in the training data is **570 tokens** (p10 377, p90 1072). **Giving a model whose median thought needs 570 tokens a budget of 16–160 tokens makes closure physically impossible.**

Splitting the main run by whether it was budget-truncated (the criterion is $n_{\text{new}} = \text{max_new}$, which is exact for single-turn items):

Table 14. Leak rate split by budget truncation

Subset	Items	Leak rate	Criterion passed
Single-turn, budget-truncated	**644**	**82.1%**	19
Single-turn, **not truncated**	**235**	**0.0%**	47
Multi-turn (conservative lower bound)	133	0.8%	16

All 530 items judged "leaking" fall inside the truncated subset, and the 243 never-truncated items leak not once. The raw leak rate of 52.0% is therefore **not a number about the model**.

Note. This conclusion was first raised by `docs/17`. We re-verified it with an **independent script** (`scripts/audit_leak_truncation.py`, read-only, joining the raw jsonl to the item bank by id) and the **single-turn numbers agree exactly** (644 / 82.1%, 235 / 0.0%, raw 52.0%, `max_new` 16–160).

The code side has been fixed: `bench/runner.py` now records `truncated` / `max_new` / `think_leak_untruncated`, `bench/analysis.py` emits the truncation split (returning `None` rather than 0 when an old result file lacks the `truncated` field), and `scripts/bench_v3.py` gained a `--budget` override for large-budget reruns.

8.3 The Completed Comparison

`rp` / `v10` / `v13` / `v14 checkpoint-10000`, on the same 1,020-item bank, the same seed 42 and the same decoding parameters:

Table 15. Comparison of the completed models

Model	Weighted score	Raw leak	Truncated	Untruncated	Untrunc. leak
`v13`	**0.1049**	52.0%	740 (**72.5%**)	280	**0.0%**
`v14 checkpoint-10000`	**0.0829**	46.8%	665 (**65.2%**)	**355**	0.6%

Note. The truncation criterion is uniformly $n_{\text{new}} \geq \text{max_new} \times \text{turns}$ (exact for single-turn, a conservative lower bound for multi-turn); the 644 / 82.1% of the single-turn rows in Section 8.2 is that same criterion restricted to the single-turn subset. A looser rule without the $\times$turns factor would inflate the v13/v14 truncation rates to 76.2% / 69.1% respectively — **this report does not use those numbers**, and everything below uses the strict criterion.

Reading only those two rows suggests "v14 got worse". But they are **not** comparable quantities — the two models' "untruncated subsets" are **not the same items** (v13 stops only on the 280 easier-to-finish items, v14 stops on 355). An **intersection comparison** is unavoidable:

Table 16. Intersection comparison

Subset	n	v13	v14	Δ
All 1,020 items	1020	0.0814	0.0716	-0.0098
Intersection: untruncated for both	**242**	**0.1942**	**0.1405**	-0.0537
Only v13 untruncated	38	0.211	0.105	-0.106
Only v14 untruncated	113	0.097	0.044	-0.053
Both truncated	627	0.027	**0.048**	$+0.021$

Conclusions (with caveats):

1. **v14's stopping behaviour really did improve** — under the strict criterion the truncation rate falls 72.5% $\rightarrow$ 65.2% and the raw leak 52.0% $\rightarrow$ 46.8%. This agrees in direction with `docs/17`'s judgement that the firing rate is the real defect.
2. **But on the same items v14's accuracy is lower** (0.194 $\rightarrow$ 0.141, -28% relative), and three of the four subsets agree in direction.
3. **The statistical power is insufficient.** On 242 items the binomial standard error at $p \approx 0.17$ is about 0.024 and the standard error of the difference about 0.034, so $\Delta = -0.054$ is about $\mathbf{1.6}\sigma$. That *suggests* a real regression but cannot settle it. Settling it needs multiple seeds (only seed 42 exists).

8.4 A Causal Hypothesis Awaiting Test: Restarting the LR After Annealing

v14's configuration restarted a full cosine schedule **on v13 weights that had already annealed to `min_lr`**: the peak was 4×10^{-5} while v13's final LR was 1.1×10^{-5} — **a 3.6× jump**. (The rationale at the time is in the comment in `tasks/v14_272m_2k_cont.json`: "the model had already annealed to `min_lr`; restarting at 1.1×10^{-4} would break it first", so the peak was lowered to 4×10^{-5}.)

But two facts are observed together:

- v14's **training loss is lower** (2.4861 → 2.3686, against v13's 2.3707 at step 10100);
- v14's **benchmark accuracy is lower** (0.194 → 0.141 on the intersection).

"Falling training loss with falling task accuracy" is the classic signature of **overfitting / catastrophic forgetting**, and "restarting the LR after annealing" is a known trigger for it.

We have not verified this; it is listed in Section 9 as the first experiment for the next round: same data, same step span, but **without restarting the LR** (continuing from 1.1×10^{-5}), to see whether the intersection accuracy returns to the v13 level.

9 Limitations and Future Work

1. **Capacity.** The CoT capability threshold recorded in `docs/04` is 0.5–1B parameters, and 272M is below it. Long-context retrieval (fetching one fact across 1.2K tokens) and multi-step arithmetic are stably zero on 272M, and they are also zero for v10 (0.28 epoch) — these two **do not look like a training-volume problem**. But the converse cannot be excluded either: this model has **never** seen the 552M-token corpus once through (v13 reached 18.5%; v14 accumulates about 92.6% by step 50,000). Neither direction currently has decisive evidence.
2. **Memory.** Physical batch 2 requires 9.81 GiB reserved, which does not fit on an 8 GB card; activations are 59% of peak memory and are the only movable part. **The 8 GB limit constrains throughput, not capacity** — per the estimate in `docs/16` (**not independently verified**; see the reference-reliability note below), the same card fits roughly 700M parameters once gradient checkpointing is enabled, at a cost of 3.3× per step.
3. **Train/inference inconsistency.** Training takes the chunkwise path while `generate()`'s prefill takes the parallel path; the two are not numerically identical, and there is currently **no guard** for it.
4. **Unverified citations.** See the annotations in the reference list.
5. **An unresolved capability regression.** v14 is 28% below v13 on the truncation-corrected intersection (0.194 → 0.141, about 1.6σ). Only a single seed exists, so **the statistical power is insufficient to conclude**, but the direction is consistent and is accompanied by the typical "falling training loss with falling task accuracy" signature. The leading hypothesis awaiting test is the **restart of the learning rate after annealing** of Section 8.4.
6. **One unexplained observation.** v13's loss near step 10020 is ≈ 2.47, while v14 at the same position is 2.4092 — a gap of about 0.06, far larger than the measured chunk-64/128 single-batch difference of 2.4e-7. The two should have seen the same data window (v14 was rolled back to `checkpoint-10000` and rerun). **We have not got to the bottom of it**; candidate explanations include the dataloader's skip semantics, and the effect of the first leg's abnormal termination on the RNG state.

10 Conclusion

On a single 8 GB consumer GPU the per-optimization-step time of RetNet-272M falls from **2176 ms to 662 ms (3.29×)**, with every conclusion built on wall-clock measurement of the real `Trainer` path and on numerical-equivalence tests. The main algorithmic contribution of this report is a **closed-form state scan**:

exploiting the fact that the per-chunk decay is constant across the loop, it replaces the n_c-step serial recurrence with a single batched matrix multiplication, cutting each layer from 1127 μs to 190 μs while the relative error stays at the level of fp32 accumulation noise.

Three methodological conclusions matter more than the numbers:

1. **A claim that "we are at the limit" must be traceable to its measurement conditions.** The project inferred "removing kernels is useless" from a battery-measured "20.8% GPU idle", yet on AC the true occupancy is 94% — and both real optimizations (8% combined) are precisely kernel removal.
2. **The shape of the bottleneck changes.** Before the optimizations it was launch count; after them it is the shape-limited GEMM ceiling. The same model on the same hardware, and a completely different answer.
3. **Silent correctness defects cost more than performance problems.** All three defects reported here (checkpoints that would not load, a resume path that never worked, an equivalence test that compared a function with itself) raised no error. They survived this long only because **nobody ever actually loaded those checkpoints**.

A. Appendix A: Reproduction

The command block is reproduced verbatim from the source (only the # comments have been translated):

```
# 1) Step-time comparison (real Trainer path, AC power)
$env:RETNET_FAST_CHUNKWISE='1'
python scripts\bench_trainer.py --model_dir checkpoints\retnet_272m_rp `
    --data_dir data_cache\chat_pack_v13 --max_steps 70 --batch_size 1 `
    --grad_accum 4 --chunk_size 128 --compile --no_grad_checkpoint `
    --optim adamw_bnb_8bit --warm_skip 25

# 2) chunk sweep (only meaningful after fixing config.chunk_size)
#    replace --chunk_size with 64 / 128 / 256 in turn

# 3) Kernel-level attribution (chrome trace -> aggregate by CUDA kernel)
python scripts\profile_kernels.py --batch_size 1 --grad_accum 4 --chunk_size 128 `
    --compile --no_grad_checkpoint --optim adamw_bnb_8bit --steps 10 --active 3

# 4) How far GEMM is from the ceiling
python scripts\gemm_ceiling.py --reps 25

# 5) Equivalence regression (mandatory)
python tests\test_fast_chunkwise.py      # 15 combinations
python tests\test_head_group_norm.py
python tests\test_control.py             # external flag control, 22 cases

# 6) Checkpoint health check
python scripts\check_checkpoints.py --root checkpoints          # key names, O(1)
python scripts\verify_checkpoint_loads.py --ckpt <dir>          # really run one forward

# 7) Leak rate / truncation split (read-only, no GPU needed)
python scripts\audit_leak_truncation.py --run logs\bench_v3\v13.T0.7_s1.jsonl --by-cat

# 8) Evaluation
python scripts\bench_v3.py --models "m=checkpoints/retnet_272m_v14/checkpoint-10000" `
    --tokenizer data_cache/chat_pack_v13/bpe_tokenizer.json --seeds 42 `
    --out logs\bench_v3 --tag <tag>
# Large-budget rerun (deciding "will it close" needs a bigger budget, see Section 8.2)
#    append --budget 1024

# 9) Training (Laptop; schtasks detaches it from SSH)
schtasks /create /tn RetNet_v14r /tr "cmd /c C:\Users\Ayaka\RetNet\tasks\start_v14_resume.bat" `
    /sc once /st 23:59 /f
schtasks /run /tn RetNet_v14r
# Graceful stop (do not taskkill):
python run_task.py --ctrl next-checkpoint
```

B. Appendix B: Key Code

The snippets below keep the original code and the original Chinese comments (the shared preamble supports CJK inside `lstlisting`).

B.1 Closed-Form State Scan (`retnet_hf/retention_fast.py`)

```
# (2) 闭式扫描: decay = chunk_decay 在循环里是常量, 于是
#     S_prev[i] = d^i·state + Σ_{j<i} d^(i-1-j)·U_j
# 这正是对 T[h,i,j] = d_h^(i-1-j)（下三角）的一次批量矩阵乘。
# 多出来的一行 i=nc 恰好是 new_state。所有指数非负 → 无 γ^-j 溢出。
log_d = torch.log(gamma.to(device=dev, dtype=torch.float32)) * float(chunk_size)
idx = torch.arange(nc + 1, device=dev)
expo_t = idx[:, None] - torch.arange(nc, device=dev)[None, :] - 1
valid_t = expo_t >= 0
T = torch.exp(log_d[:, None, None] * expo_t.clamp_min(0).to(torch.float32)[None])
T = torch.where(valid_t[None], T, torch.zeros((), device=dev, dtype=torch.float32))
T = T.to(q.dtype)                                          # [H, nc+1, nc]
```

```
U_flat = U.reshape(B, H, nc, kd * vd)
S_all = torch.matmul(T[None], U_flat)                    # [B,H,nc+1,kd*vd]
if has_state:
    t_state = torch.exp(log_d[:, None] * idx.to(torch.float32)[None]).to(q.dtype)
    S_all = S_all + t_state[None, :, :, None] * state.reshape(B, H, 1, kd * vd)

S_prev = S_all[:, :, :nc].reshape(B, H, nc, kd, vd)
new_state = S_all[:, :, nc].reshape(B, H, kd, vd)
```

B.2 Log-Space Decay Mask (`retnet_hf/retention_fast.py`)

```
causal, expo, relf = _index_tables(chunk_size, device)
g = gamma.to(device=device, dtype=torch.float32)
log_gamma = torch.log(g)

# exp(log γ · delta): 两个方向都正确饱和。绝不能写成 γ^i · γ^-j ——
# 本模型 γ 最小 0.668，0.668^-255 = 1e44 > 3.4e38 → Inf → NaN。
inner_mask = torch.exp(log_gamma[:, None, None] * expo[None])
inner_mask = torch.where(causal, inner_mask,
                         torch.zeros((), device=device, dtype=torch.float32))

pow1d = torch.exp(log_gamma[:, None] * relf[None, :])     # [H,C] = γ^j
cross_decay = pow1d * g[:, None]                          # γ^(j+1)
value_decay = pow1d.flip(-1)                              # γ^(C-1-j)
chunk_decay = pow1d[:, -1] * g                            # γ^C
```

B.3 In-Place Compilation (`retnet_hf/train.py`)

```
# 就地替换 forward，**不要**把模型包成 OptimizedModule —— 那会给 state_dict
# 的每一个键加 `_orig_mod.` 前缀，让保存的检查点静默变成随机模型。
compiled_fwd = torch.compile(trainer.model.forward, backend="inductor")
trainer.model.forward = compiled_fwd
```

B.4 Export Self-Check (`retnet_hf/train.py`)

```
def verify_export(path) -> bool:
    """safetensors 的键里不能有 `_orig_mod.`，否则 from_pretrained 会静默随机初始化。"""
    ...
    bad = [k for k in keys if k.startswith("_orig_mod.")]
    if bad:
        print(f"[verify] ✗ 导出里有 {len(bad)}/{len(keys)} 个键带 _orig_mod. 前缀 —— "
              f"这个检查点用 from_pretrained 加载会**静默变成随机模型**。")
        return False
    print(f"[verify] ✓ 导出键名干净（{len(keys)} 个），可直接 from_pretrained")
```

B.5 Resume Path (`retnet_hf/train.py`)

```
if args.resume:
    from transformers.trainer_utils import get_last_checkpoint
    last = get_last_checkpoint(args.output_dir)
    if last is None:
        raise SystemExit(f"--resume 但 {args.output_dir} 里找不到 checkpoint-* 子目录。")
    print(f"[resume] 从 {last} 续训（优化器 / LR 调度器状态由 HF 一并恢复）", flush=True)
    resume_arg = last
else:
    resume_arg = False
trainer.train(resume_from_checkpoint=resume_arg)
```

C. Appendix C: External Flag Control (`retnet_hf/control.py`)

Stopping a running training job used to require `taskkill`, and even the order had to be right (`run_task.py` does `if rc != 0 and attempts < auto_restart_max: restart` — killing `train.py` directly yields `rc=1` and gets the job restarted). A flag lets training exit **gracefully at a step boundary**:

- `run` — continue; the default, also used to clear an instruction.
- `stop` — stop after the current optimization step, without waiting for a checkpoint.
- `next-checkpoint` — stop after the next checkpoint write; the checkpoint is flushed to disk before the exit.
- `save` — write a checkpoint immediately, then continue.

The correctness basis: in `transformers/trainer.py`'s `_inner_training_loop`, line 2755 `on_step_end` precedes line 2756 `_maybe_log_save_evaluate` (checkpoint + evaluation) and the `break` is at line 2772. Setting `control.should_training_stop` inside `on_step_end` therefore **saves the checkpoint before exiting** — which is exactly why "stop after the next checkpoint" can be written correctly.

`stop` and `next-checkpoint` are **sticky** (the flag is not cleared after it fires), so that `run_task.py` can tell "stopped on external request" apart from "crashed, should be restarted".

D. Appendix D: Defect Summary

#	Defect	Symptom	Why silent	Fix
1	`_orig_mod.` prefix	loss = nan after loading a checkpoint	`from_pretrained` silently drops the unexpected keys	in-place forward compile + strip tool + export self-check
2	Resume path	`Can't find a valid checkpoint`	the path was never actually exercised	`get_last_checkpoint()`
3	Self-comparing equivalence test	always reports "bit-identical"	the environment variable forwards the reference to the function under test	`reference_path` context plus a relative criterion
4	HF loss normalisation	loss inflated `accum`×	it only fires when `accum > 1`	`accepts_loss_kwargs = False`
5	`config.chunk_size`	the config is ignored	nothing errors	fall back to the config in `forward`
6	`eval_batch_mult`	the config is ignored and evaluation is 2.4× slower	nothing errors	wire it into `train.py`
7	`torch.compile` silently ineffective	the log says applied, the speedup is 0%	inductor compiles lazily plus the wrapping is misplaced	an explicit `import triton` + in-place forward replacement
8	`save_total_limit`	the config says 4, the actual value is 3, and `checkpoint-10000` was rotated away	the deletion line is `logger.info` while `transformers` defaults to `WARNING`	wire it into `train.py`; the criterion is to read `training_args.bin`, not the config JSON

1. References

[1] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, X. Zhang, J. Li, F. Wei. *Retentive Network: A Successor to Transformer for Large Language Models.* arXiv:2307.08621, 2023.

[2] S. Yang, B. Wang, Y. Shen, R. Panda, Y. Kim. *Gated Linear Attention Transformers with Hardware-Efficient Training.* arXiv:2312.06635, 2023.

[3] A. Katharopoulos, A. Vyas, N. Pappas, F. Fleuret. *Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention.* arXiv:2006.16236, 2020.

[4] A. Gu, T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv:2312.00752, 2023.

[5] T. Dao, A. Gu. *Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality.* arXiv:2405.21060, 2024.

[6] S. Yang, B. Wang, Y. Zhang, Y. Shen, Y. Kim. *Parallelizing Linear Transformers with the Delta Rule over Sequence Length.* arXiv:2406.06484, 2024.

[7] S. Yang, J. Kautz, A. Hatamizadeh. *Gated Delta Networks: Improving Mamba2 with Delta Rule.* arXiv:2412.06464, 2024.

[8] S. De, S. L. Smith, A. Fernando, A. Botev, G. Cristian-Muraru, A. Gu, et al. *Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models.* arXiv:2402.19427, 2024.

[9] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, et al. *RWKV: Reinventing RNNs for the Transformer Era.* arXiv:2305.13048, 2023.

[10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* arXiv:2205.14135, 2022.

[11] T. Dettmers, M. Lewis, S. Shleifer, L. Zettlemoyer. *8-bit Optimizers via Block-wise Quantization.* arXiv:2110.02861, 2021.

[12] P.-L. Hsu, Y. Dai, V. Kothapalli, Q. Song, S. Tang, S. Zhu, et al. *Liger Kernel: Efficient Triton Kernels for LLM Training.* arXiv:2410.10989, 2024.

[13] I. Loshchilov, F. Hutter. *Decoupled Weight Decay Regularization.* arXiv:1711.05101, 2017.

[14] I. Loshchilov, F. Hutter. *SGDR: Stochastic Gradient Descent with Warm Restarts.* arXiv:1608.03983, 2016.

[15] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, et al. *Mixed Precision Training.* arXiv:1710.03740, 2017.

[16] B. Zhang, R. Sennrich. *Root Mean Square Layer Normalization.* arXiv:1910.07467, 2019.

[17] Y. Wu, K. He. *Group Normalization.* arXiv:1803.08494, 2018.

[18] T. Chen, I. Goodfellow, J. Shlens. *Net2Net: Accelerating Learning via Knowledge Transfer.* arXiv:1511.05641, 2015.

[19] C. Chen, Y. Yin, L. Shang, X. Jiang, Y. Qin, F. Wang, et al. *bert2BERT: Towards Reusable Pretrained Language Models.* arXiv:2110.10329, 2021.

[20] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, et al. *Training Compute-Optimal Large Language Models.* arXiv:2203.15556, 2022.

[21] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv:2501.12948, 2025.

[22] W. Merrill, A. Sabharwal. *The Illusion of State in State-Space Models.* arXiv:2404.08819, 2024.

[23] O. Lieber, B. Lenz, H. Bata, G. Cohen, J. Osin, I. Dalmedigos, et al. *Jamba: A Hybrid Transformer-Mamba Language Model.* arXiv:2403.19887, 2024.

[24] P. Tillet, H.-T. Kung, D. Cox. *Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations.* MAPL@PLDI, 2019.

2. Reference Reliability Note

The following entries are quoted from internal project surveys and were **not independently verified**; confirm them before relying on them:

- **arXiv:2507.06457** (a 340M×20B controlled study of the linear:full attention ratio and recall) and **arXiv:2605.16640** (chain-of-thought solvability of hybrid vs pure linear), both cited by {docs/18}. The identifier form of the latter is suspicious (2605 would be May 2026) and should be checked.
- {docs/18}'s quantitative claim that four linear mechanisms differ by only 1.6% downstream at 340M/20B.
- {docs/16}'s estimate that 8\,GB fits 54 layers ≈ 700M parameters under gradient checkpointing.

For references **[1]–[24]**, every entry that carries a specific numeric conclusion has that conclusion **measured by this project itself** (the provenance is in the body of the report); the literature is cited only to identify the origin of the method.